Course: 04733 AIOps **September 3, 2026**

Carnegie Mellon University Africa

Name: Martin Koome

Course: 04733 AIOps, Fall 2026

Lab 1

Introduction to Kubernetes and Istio

1. Introduction

This report documents the completion of Lab 1, which covers deploying the Online Boutique microservices application to a local Kubernetes cluster, instrumenting it with Istio, and using Kiali to observe both normal traffic and artificially injected network faults.

2. Week 1 — Kubernetes and Istio Setup

2.1. Task 1: Cluster and Istio Installation

A single-node Kubernetes cluster was created on an Azure VM, and `kubectl` was configured. Istio was installed with `istioctl`, followed by the Gateway API CRDs and the observability add-ons (Prometheus, Grafana, Kiali, Jaeger) via `kubectl apply -f samples/addons/`. Figure 1 confirms all deployments in the `istio-system` namespace reached `READY 1/1`.

```
azureuser@AIopsVM:~$ kubectl get deploy -n istio-system
NAME                   READY   UP-TO-DATE   AVAILABLE   AGE
grafana                1/1     1            1           9d
istio-egressgateway    1/1     1            1           9d
istio-ingressgateway   1/1     1            1           9d
istiod                 1/1     1            1           9d
jaeger                 1/1     1            1           9d
kiali                  1/1     1            1           9d
prometheus             1/1     1            1           9d
azureuser@AIopsVM:~$
```

Figure 1: Istio control-plane and observability add-on deployments (`istiod`, ingress/egress gateways, Prometheus, Grafana, Kiali, Jaeger) in the `istio-system` namespace, all reporting `READY 1/1`.

2.2. Task 2: Deploying the Boutique Application

The `microservices-demo` repository was cloned, the provided Kustomize overlay files were placed under `kustomize/base`, and the application was deployed with `kubectl apply -k kustomize/`. All pods reached `Running 2/2` (application container plus Envoy sidecar), and all deployments reported `1/1` availability (Figures 2 and 3). The Frontend service was then exposed locally with `kubectl port-forward deployment/frontend 8080:8080`, and the homepage was verified in the browser (Figure 4).

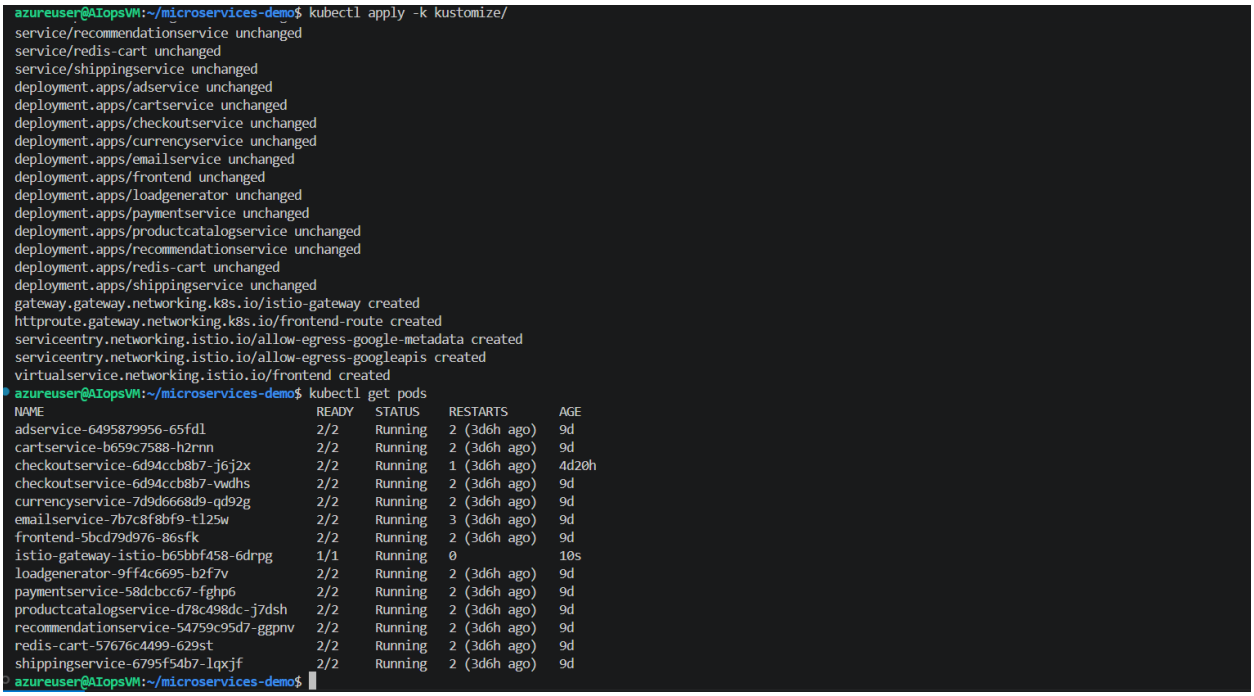


Figure 2: Boutique application pods running with `2/2` containers per pod (application container + Envoy sidecar) after applying the Kustomize manifests.

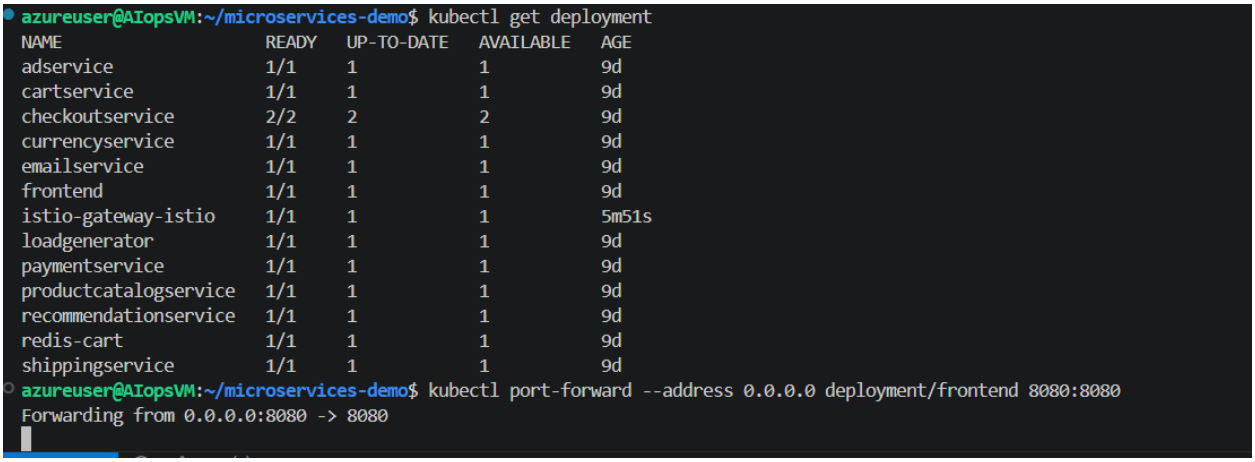


Figure 3: Boutique deployments in the `default` namespace all reporting `1/1` availability, with the Frontend service port-forwarded to `localhost:8080`.

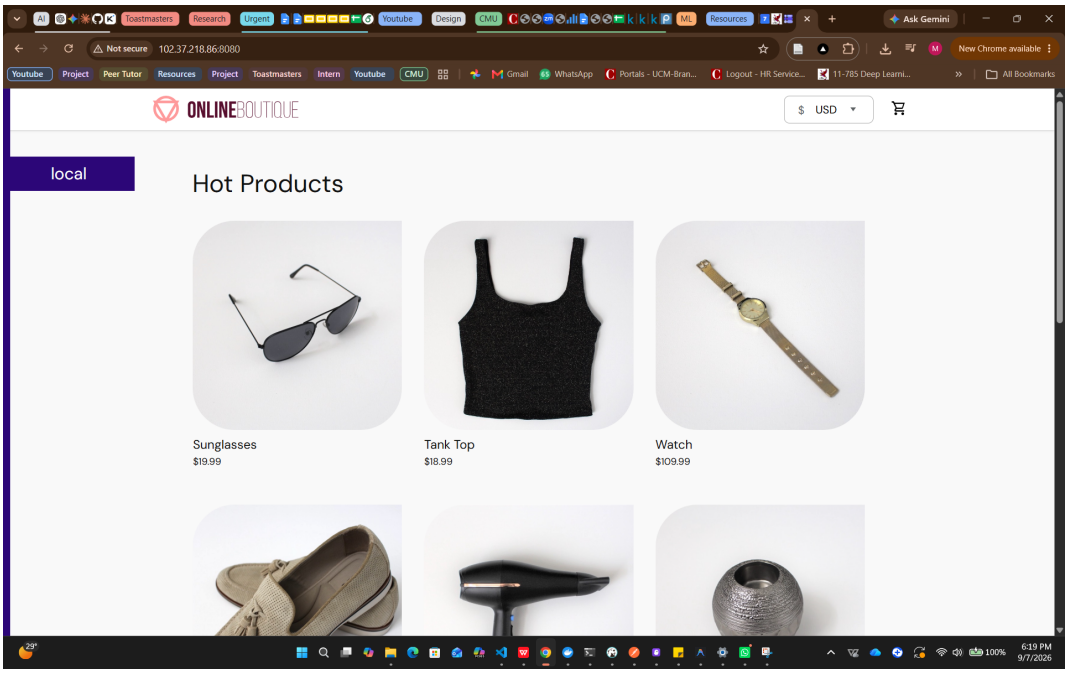


Figure 4: Online Boutique homepage rendered at `localhost:8080`, confirming the Frontend service is reachable through the port-forward tunnel.

2.3. Task 3: Rebuilding and Redeploying the Frontend Service

The Frontend Docker image was rebuilt locally with `docker build -t frontend .`, and the `image` field in `frontend.yaml` was updated to `frontend:latest`. The deployment was reapplied with `kubectl apply -f frontend.yaml`, triggering a rolling update. Figure 5 shows `kubectl describe deploy/frontend` confirming the new ReplicaSet is running the locally built image.

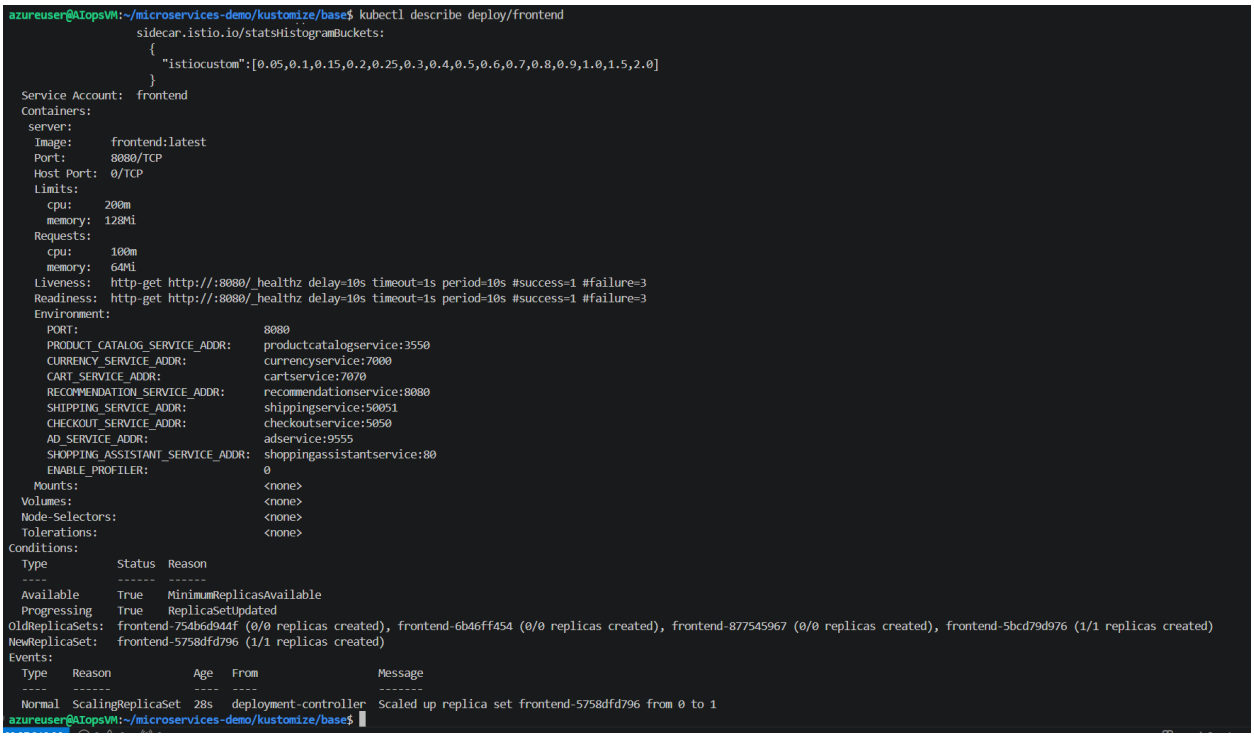


Figure 5: Output of `kubectl describe deploy/frontend` after the rolling update, confirming the active pod uses the locally built `frontend:latest` image.

3. Week 2 — Observability and Fault Injection with Istio

3.1. Task 1: Baseline Traffic Monitoring with Kiali

Kiali was launched with `istioctl dashboard kiali`. The top-level Traffic Graph was configured to display average response times with a 10 s refresh (Figure 6), and the detailed Shippingservice/Checkoutservice view was set to the *Source* perspective to establish a healthy baseline before any faults were introduced (Figure 7).

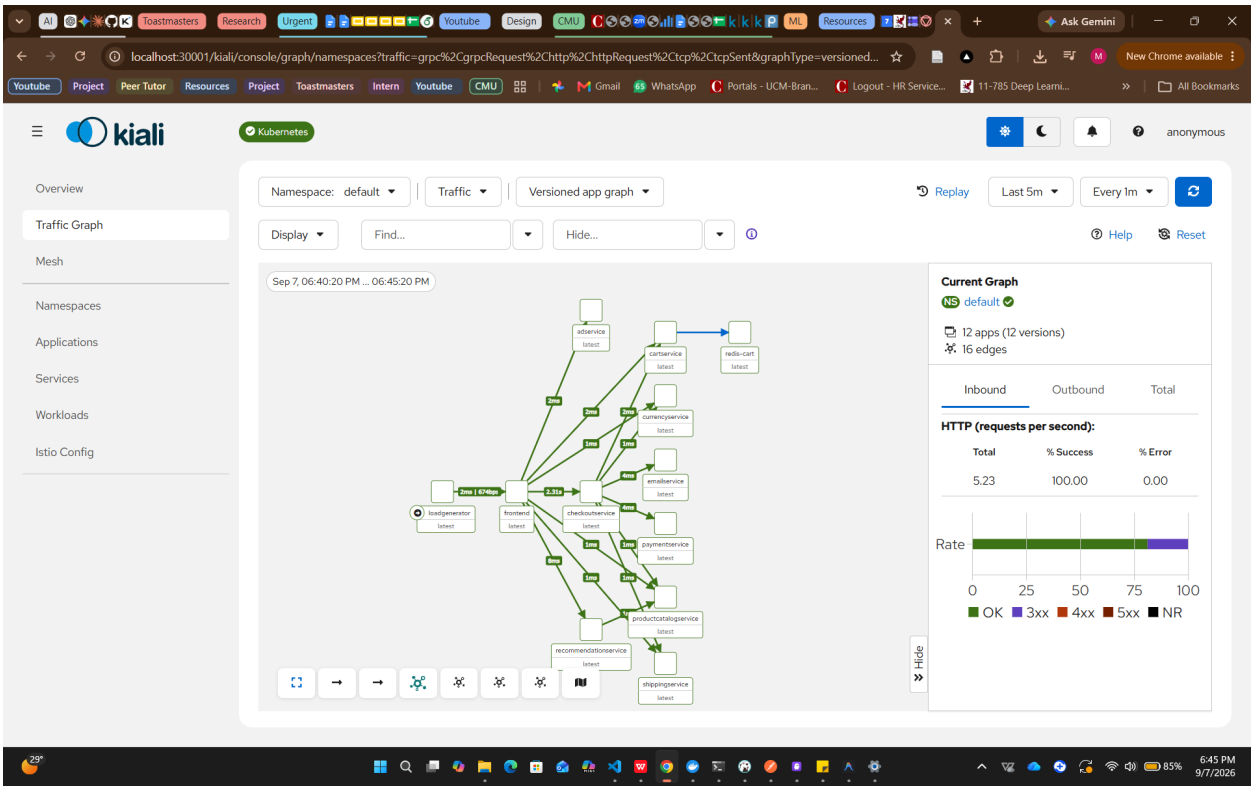


Figure 6: Top-level Kiali traffic graph for the `default` namespace showing healthy request flow across all Boutique microservices.

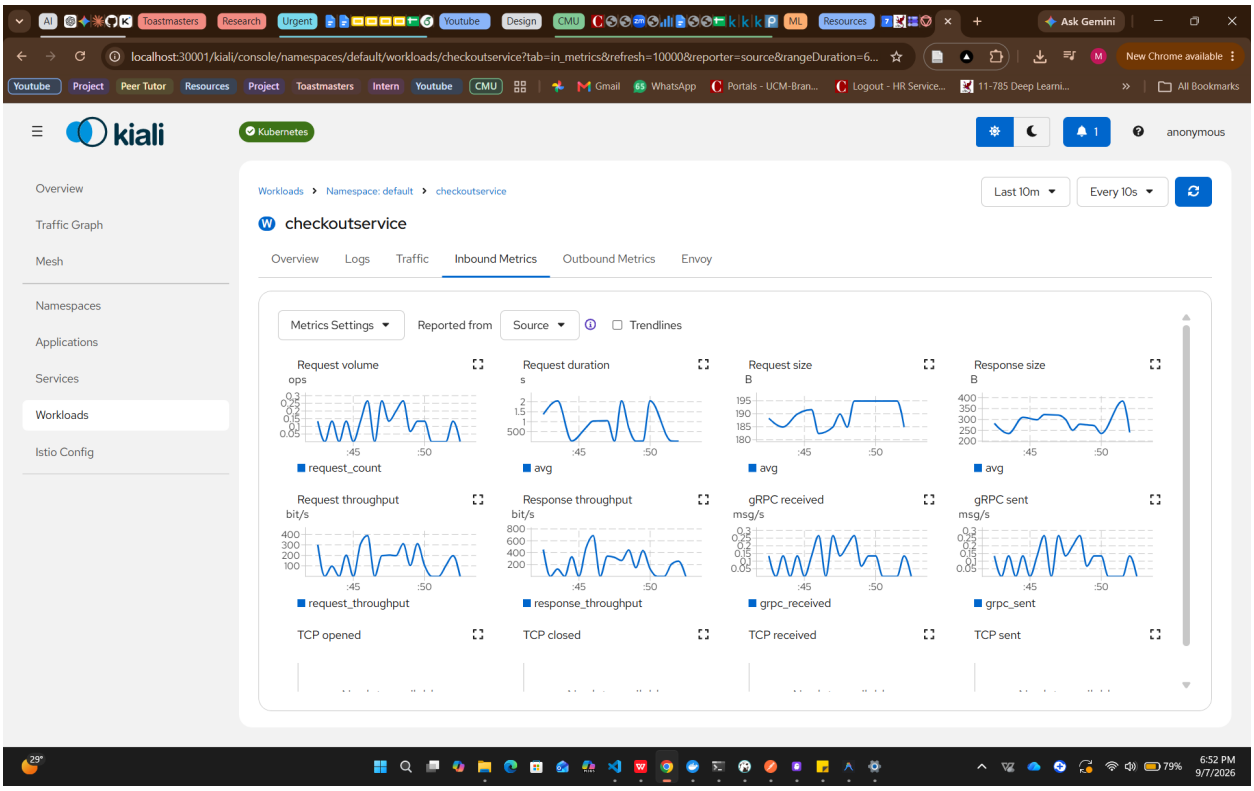


Figure 7: Baseline (pre-fault-injection) inbound metrics for the Checkoutservice/Shippingservice path, showing normal, oscillating request-duration values with no added latency.

3.2. Task 2: Selective Fault Injection (Checkoutservice → Shippingservice)

An Istio `VirtualService` fault-injection rule was applied to introduce a fixed 1 s delay for calls to `shippingservice` that originate specifically from `checkoutservice` (matched via `sourceLabels: app: checkoutservice`), leaving calls from `frontend` unaffected. The applied configuration was confirmed with `kubectl get virtualservice shippingservice -o yaml` (Listing 1), and the resulting flattened response-time trace in Kiali is shown in Figure 9.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: shippingservice
  namespace: default
spec:
  hosts:
  - shippingservice
  http:
  - fault:
      delay:
        fixedDelay: 1s
        percentage:
          value: 100
```

```
    match:
    - sourceLabels:
        app: checkoutservice
    route:
    - destination:
        host: shippingservice
  - route:
    - destination:
        host: shippingservice
```

Listing 1: Applied `shippingservice` VirtualService with a 1 s fault-injection rule scoped to `checkoutservice` traffic.



Figure 8: Terminal confirmation of the applied `VirtualService` fault configuration via `kubectl get virtualservice shippingservice -o yaml`.

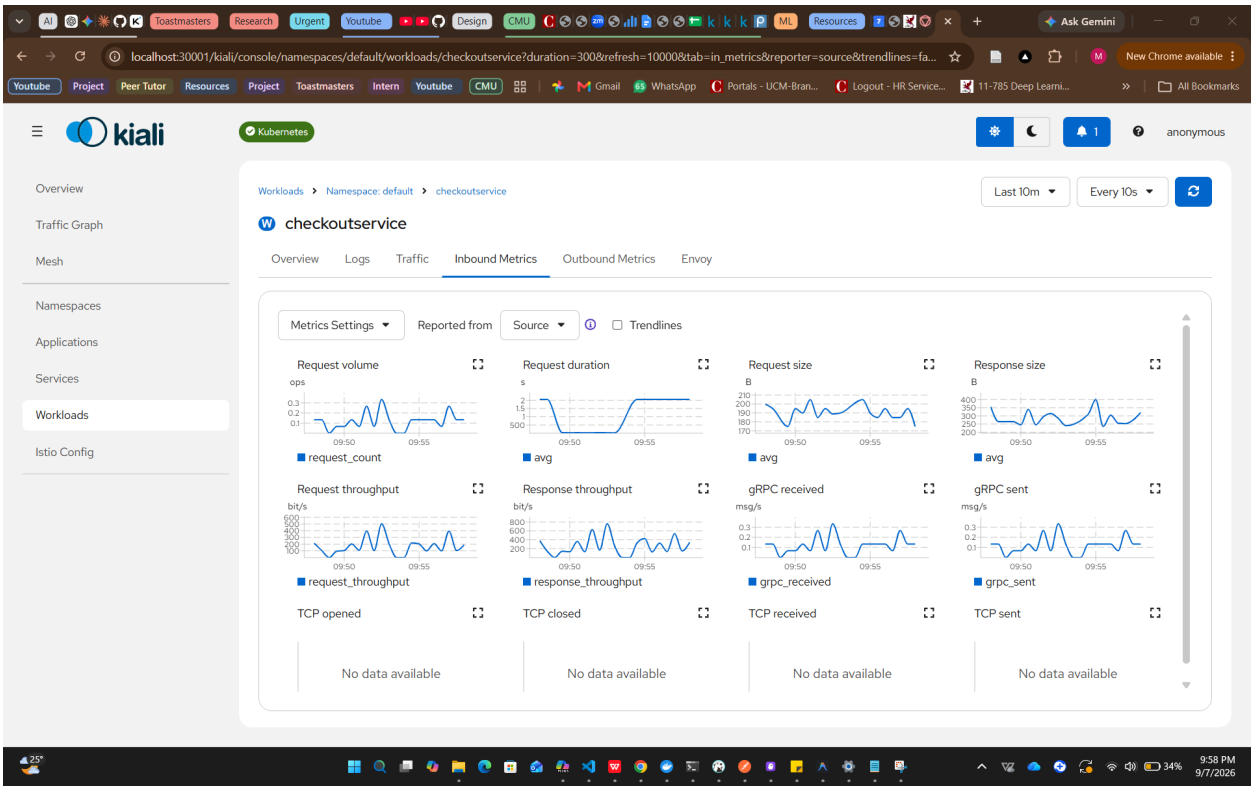


Figure 9: Inbound metrics after applying the 1 s fault-injection rule: the request-duration trace flattens at approximately 1.5 s during the affected window, reflecting the added delay on Checkoutservice-originated calls.

3.3. Task 3: Pod-Specific Fault Injection with a Scaled Deployment

The `checkoutservice` deployment was scaled to two replicas with `kubectl scale deploy checkoutservice --replicas=2`. One of the two pods was identified with `kubectl get pods` and edited with `kubectl edit pod <podID>` to add a custom label. A second `VirtualService` fault rule was then written for `shippingservice`, matching on `sourceLabels: app: checkoutservice` together with the newly added pod label, so that only calls from the labeled pod incur the 1 s delay.

Since only one of the two `checkoutservice` pods now incurs the delay when calling `shippingservice`, the aggregate response time observed in Kiali settles near 500 ms on average, reflecting the mix of delayed and un-delayed requests (Figure 10).

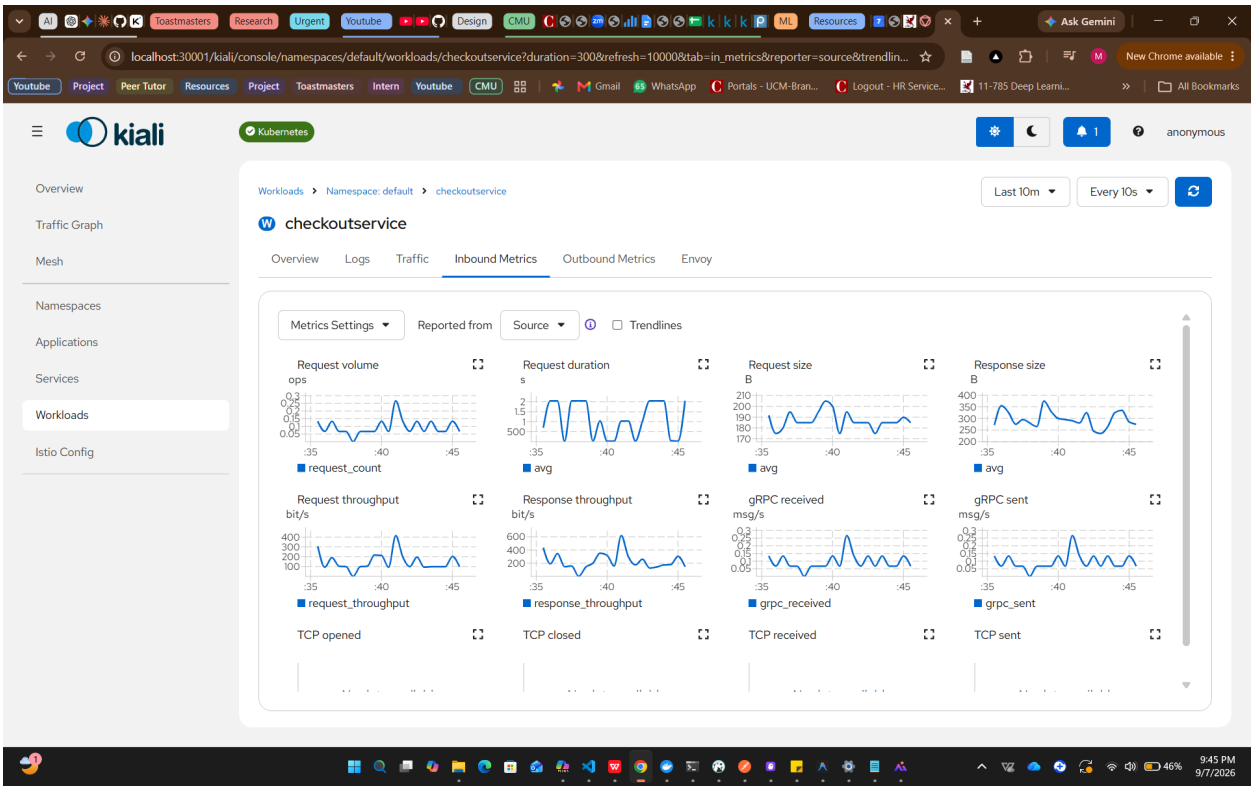


Figure 10: Inbound metrics after scaling `checkoutservice` to two replicas and applying the pod-specific fault rule; the averaged response time settles near 500 ms, consistent with only one of the two pods incurring the added 1 s delay.

4. Conclusion

This lab provided hands-on experience deploying a multi-service application to Kubernetes, performing a rolling update of a service image, and using Istio and Kiali to both observe baseline microservice traffic and inject targeted network faults first at the service level, then at the level of an individual pod via label-based routing rules.